Guía de Estudio para Quiz de Laboratorio

ESP32-S3 y Periféricos de Bioinstrumentación

Introducción a la Bioinstrumentación

Evaluación Individual de Laboratorio

Objetivo: Demostrar manejo de entradas/salidas, periféricos y comunicación serial en el ESP32-S3 usando Arduino IDE.

Tabla de Contenido

Tabla de Contenido 2
1. Tabla de Pines Oficiales del Montaje 3
2. Bibliotecas Necesarias..... 3
3. Códigos por Tema 4
4. Lo Más Importante para Memorizar Antes del Quiz 28
5. Lo que Más Probablemente Pueden Pedir Cambiar 28
6. Sección de Repaso Rápido..... 30

1. Tabla de Pines Oficiales del Montaje

Elemento	Pin / Conexión
LED externo	GPIO4, con resistencia limitadora (220 Ω) y conexión a GND
Pulsador	GPIO5 usando INPUT_PULLUP, con el otro terminal a GND
Potenciómetro 1	Señal al GPIO6; extremos a 3.3 V y GND
Potenciómetro 2	Señal al GPIO7; extremos a 3.3 V y GND
OLED SSD1306	I2C: SDA en GPIO8 y SCL en GPIO9; alimentación y GND
NeoPixel	GPIO48 si es externo, o LED RGB integrado de la tarjeta
Computador	USB / Monitor Serie (115200 baud)

2. Bibliotecas Necesarias

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_NeoPixel.h>
```

Biblioteca	Función
Wire.h	Comunicación I2C (necesaria para la OLED)
Adafruit_GFX.h	Funciones gráficas básicas (texto, formas, píxeles)
Adafruit_SSD1306.h	Control específico de la pantalla OLED SSD1306
Adafruit_NeoPixel.h	Control del LED RGB NeoPixel (colores, brillo)

3. Códigos por Tema

1. LED externo en GPIO4: Parpadeo simple

Posible pregunta: Haga que el LED conectado a GPIO4 parpadee cada 500 ms.

Código completo:

```
#define LED_PIN 4

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);   // LED encendido
    delay(500);
    digitalWrite(LED_PIN, LOW);    // LED apagado
    delay(500);
}
```

Explicación: GPIO4 se configura como OUTPUT en setup(). En loop(), se alterna entre HIGH (encendido) y LOW (apagado) con delay(500) entre cada estado.

Cambio rápido en el quiz: Cambiar delay(500) por otro valor: delay(200) = rápido, delay(1000) = lento.

Frase para el profesor: Este código configura GPIO4 como salida y cambia su estado entre HIGH y LOW cada 500 ms para producir el parpadeo.

2. LED externo controlado por pulsador

Posible pregunta: Encienda el LED de GPIO4 mientras el botón de GPIO5 esté presionado.

Código completo:

```
#define LED_PIN  4
#define BTN_PIN  5

void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);
}

void loop() {
    int boton = digitalRead(BTN_PIN);

    if (boton == LOW) {
        digitalWrite(LED_PIN, HIGH);  // Botón presionado → LED encendido
    } else {
        digitalWrite(LED_PIN, LOW);   // Botón suelto → LED apagado
    }
}
```

Explicación: Con INPUT_PULLUP, el pin internamente se conecta a 3.3 V. Al presionar el botón (que va a GND), la lectura cae a LOW. Por eso presionado = LOW, suelto = HIGH.

Cambio rápido en el quiz: Invertir lógica: cambiar == LOW por == HIGH para encender cuando esté suelto.

Frase para el profesor: Con INPUT_PULLUP, el botón presionado se lee como LOW porque conecta el pin a GND.

3. Pulsador por Monitor Serie

Posible pregunta: Leer el botón en GPIO5 y mostrar por Serial cuando esté presionado.

Código completo:

```
#define BTN_PIN 5

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
}
```

```
void loop() {
    if (digitalRead(BTN_PIN) == LOW) {
        Serial.println("Boton presionado");
        delay(100);  // Anti-rebote simple
    }
}
```

Explicación: Serial.begin(115200) inicia la comunicación serie. Dentro del loop se lee el estado del botón. Si es LOW (presionado), se imprime el mensaje.

Cambio rápido en el quiz: Cambiar el texto dentro de println(), o agregar un mensaje adicional cuando esté suelto (else).

Frase para el profesor: analogRead() convierte el voltaje del potenciómetro en un número entre 0 y 4095. El Serial.begin(115200) debe estar en setup().

4. Detectar solo una pulsación del botón

Posible pregunta: Mostrar el mensaje solo una vez cada vez que se presione el botón.

Código completo:

```
#define BTN_PIN 5

int estadoAnterior = HIGH;

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
}

void loop() {
    int estadoActual = digitalRead(BTN_PIN);

    if (estadoActual == LOW && estadoAnterior == HIGH) {
        Serial.println("Pulsacion detectada!");
    }
```

```
    estadoAnterior = estadoActual;
    delay(20);  // Anti-rebote
}
```

Explicación: Se guarda el estado anterior. Solo cuando el estado cambia de HIGH a LOW (flanco de bajada) se imprime el mensaje. Esto evita repeticiones mientras el botón permanece presionado.

Cambio rápido en el quiz: Cambiar la condición a estadoActual == HIGH && estadoAnterior == LOW para detectar cuando se suelta.

Frase para el profesor: Comparo el estado actual con el anterior para detectar solo el momento del cambio, no el estado continuo.

5. Potenciómetro en GPIO6: lectura ADC

Posible pregunta: Leer el potenciómetro de GPIO6 y mostrar el valor ADC.

Código completo:

```
#define POT1_PIN 6

void setup() {
    Serial.begin(115200);
}

void loop() {
    int lectura = analogRead(POT1_PIN);
    Serial.print("ADC GPIO6: ");
    Serial.println(lectura);
    delay(200);
}
```

Explicación: analogRead() no necesita pinMode(). Devuelve un entero entre 0 (0 V) y 4095 (3.3 V). El ESP32-S3 tiene ADC de 12 bits.

Cambio rápido en el quiz: Cambiar el pin a 7 para leer el potenciómetro 2. Cambiar el delay para más o menos frecuencia de lectura.

Frase para el profesor: analogRead() convierte el voltaje del potenciómetro en un número entre 0 y 4095 porque el ADC es de 12 bits.

6. Potenciómetro en GPIO6 convertido a voltaje

Posible pregunta: Leer el voltaje de GPIO6 y mostrarlo con dos cifras decimales.

Código completo:

```
#define POT1_PIN 6

void setup() {
    Serial.begin(115200);
}

void loop() {
    int lectura = analogRead(POT1_PIN);
    float voltaje = lectura * 3.3 / 4095.0;

    Serial.print("Voltaje GPIO6: ");
    Serial.print(voltaje, 2);
    Serial.println(" V");
    delay(200);
}
```

Explicación: La fórmula convierte la lectura ADC (0-4095) al rango de voltaje (0-3.3 V). El segundo parámetro de Serial.print() indica decimales.

Cambio rápido en el quiz: Cambiar la cantidad de decimales: Serial.print(voltaje, 3) para tres decimales.

Frase para el profesor: Multiplico la lectura por 3.3 y divido entre 4095 para obtener el voltaje real que entrega el potenciómetro.

7. Leer los dos potenciómetros GPIO6 y GPIO7

📋 **Posible pregunta:** Leer ambos potenciómetros y mostrar los voltajes por Monitor Serie.

Código completo:

```
#define POT1_PIN 6
#define POT2_PIN 7

void setup() {
    Serial.begin(115200);
}

void loop() {
    float v1 = analogRead(POT1_PIN) * 3.3 / 4095.0;
    float v2 = analogRead(POT2_PIN) * 3.3 / 4095.0;

    Serial.print("Pot GPIO6: ");
    Serial.print(v1, 2);
    Serial.print(" V  |  Pot GPIO7: ");
    Serial.print(v2, 2);
    Serial.println(" V");
    delay(300);
}
```

💡 **Explicación:** Dos analogRead() independientes, uno por pin. Cada lectura se convierte a voltaje con la misma fórmula.

⚡ **Cambio rápido en el quiz:** Agregar Serial.printf() para formato más compacto: Serial.printf("V1=%.2f V2=%.2f\n", v1, v2);

🗣️ **Frase para el profesor:** Cada analogRead() lee un canal ADC diferente. Los convierto a voltaje con la misma fórmula para ambos.

8. LED controlado por potenciómetro (umbral de voltaje)

📋 **Posible pregunta:** Encienda el LED si el voltaje del potenciómetro supera 1.65 V.

Código completo:

```
#define LED_PIN  4
#define POT1_PIN 6

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    float voltaje = analogRead(POT1_PIN) * 3.3 / 4095.0;

    if (voltaje > 1.65) {
        digitalWrite(LED_PIN, HIGH);
    } else {
        digitalWrite(LED_PIN, LOW);
    }

    Serial.print("Voltaje: ");
    Serial.print(voltaje, 2);
    Serial.println(" V");
    delay(100);
}
```

Explicación: Se lee el potenciómetro, se convierte a voltaje, y se compara con el umbral. Si supera 1.65 V (mitad de 3.3 V), el LED se enciende.

Cambio rápido en el quiz: Cambiar el umbral: reemplazar 1.65 por cualquier otro valor entre 0.0 y 3.3.

Frase para el profesor: Leo el voltaje del potenciómetro y lo comparo con un umbral. Si supera el umbral enciendo el LED.

9. OLED SSD1306: mostrar mensaje básico

Posible pregunta: Mostrar un texto en la pantalla OLED.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
    Wire.begin(8, 9);  // SDA=8, SCL=9
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.println("Hola ESP32!");
    display.display();
}

void loop() {
    // Nada aqui, el texto queda fijo
}
```

Explicación: Wire.begin(8, 9) inicia I2C con SDA=GPIO8 y SCL=GPIO9. display.begin() inicializa la OLED en la dirección 0x3C. Siempre terminar con display.display() para enviar el buffer a la pantalla.

Cambio rápido en el quiz: Cambiar el texto en println(), cambiar setTextSize(1) para texto pequeño o setTextSize(2) para grande.

Frase para el profesor: La OLED usa I2C, por eso se inicia con Wire.begin(8, 9). Todo lo que dibujo en el buffer solo aparece al llamar display.display().

10. OLED mostrando un potenciómetro

Posible pregunta: Mostrar en OLED el voltaje del potenciómetro en GPIO6.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define POT1_PIN     6

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
    Wire.begin(8, 9);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
    float voltaje = analogRead(POT1_PIN) * 3.3 / 4095.0;

    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.println("Pot GPIO6:");
    display.print(voltaje, 2);
    display.println(" V");
    display.display();
    delay(200);
}
```

Explicación: En cada iteración del loop se lee el potenciómetro, se borra la pantalla con clearDisplay(), se escribe el valor y se envía con display(). El clearDisplay() es clave para evitar que los valores se superpongan.

Cambio rápido en el quiz: Cambiar el pin a POT2_PIN 7 para mostrar el segundo potenciómetro. Cambiar setTextSize(1) para mostrar más texto.

🗣️ **Frase para el profesor:** Llamo clearDisplay() en cada ciclo para borrar el valor anterior antes de escribir el nuevo.

11. OLED mostrando dos potenciómetros

📋 **Posible pregunta:** Mostrar en OLED los valores de GPIO6 y GPIO7.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define POT1_PIN     6
#define POT2_PIN     7

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
    Wire.begin(8, 9);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
    float v1 = analogRead(POT1_PIN) * 3.3 / 4095.0;
    float v2 = analogRead(POT2_PIN) * 3.3 / 4095.0;

    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);

    display.setCursor(0, 0);
    display.print("Pot6: ");
    display.print(v1, 2);
    display.println(" V");

    display.setCursor(0, 20);
    display.print("Pot7: ");
    display.print(v2, 2);
    display.println(" V");
```

```
    display.display();
    delay(200);
}
```

Explicación: setCursor(0, 20) posiciona el cursor en la fila 20 píxeles desde arriba para la segunda línea. Con setTextSize(1) caben más datos en pantalla.

Cambio rápido en el quiz: Cambiar la posición vertical del cursor (segundo parámetro de setCursor) para ajustar el espaciado entre líneas.

Frase para el profesor: Uso setCursor con coordenadas Y diferentes para mostrar cada valor en su propia línea de la pantalla.

12. OLED mostrando estado del botón

Posible pregunta: Mostrar en la OLED si el botón está presionado o suelto.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define BTN_PIN      5

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
    Wire.begin(8, 9);
    pinMode(BTN_PIN, INPUT_PULLUP);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
```

```
    display.setCursor(0, 20);

    if (digitalRead(BTN_PIN) == LOW) {
        display.println("PRESIONADO");
    } else {
        display.println("SUELTO");
    }

    display.display();
    delay(100);
}
```

Explicación: Se lee el botón en cada ciclo y se muestra el texto correspondiente. Con INPUT_PULLUP, LOW = presionado.

Cambio rápido en el quiz: Cambiar los textos PRESIONADO y SUELTO por cualquier otro mensaje.

Frase para el profesor: Combino digitalRead con la OLED: el resultado del botón determina qué texto aparece en pantalla.

13. NeoPixel: encender un color fijo

Posible pregunta: Cambiar el NeoPixel a color rojo, verde o azul.

Código completo:

```
#include <Adafruit_NeoPixel.h>

#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    pixel.begin();
    pixel.setBrightness(50);   // Brillo: 0-255
    pixel.setPixelColor(0, pixel.Color(255, 0, 0));  // Rojo
    pixel.show();
}

void loop() {
    // Color fijo, nada en loop
}
```

Explicación: pixel.Color(R, G, B) define el color con componentes rojo, verde y azul (0-255 cada uno). pixel.show() envía el color al LED. pixel.setBrightness() controla la intensidad.

Cambio rápido en el quiz: Cambiar el color: pixel.Color(0, 255, 0) = verde | pixel.Color(0, 0, 255) = azul | pixel.Color(255, 255, 0) = amarillo | pixel.Color(0, 0, 0) = apagado.

Frase para el profesor: El NeoPixel se actualiza con pixel.show() después de llamar setPixelColor().

Tabla de colores NeoPixel:

Color	Código

Rojo	`pixel.Color(255, 0, 0)`
Verde	`pixel.Color(0, 255, 0)`
Azul	`pixel.Color(0, 0, 255)`
Amarillo	`pixel.Color(255, 255, 0)`
Blanco	`pixel.Color(255, 255, 255)`
Apagado	`pixel.Color(0, 0, 0)`

14. NeoPixel cambia con el botón

📋 **Posible pregunta:** Hacer que el NeoPixel sea verde cuando el botón esté presionado y rojo cuando esté suelto.

Código completo:

```
#include <Adafruit_NeoPixel.h>

#define BTN_PIN      5
#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    pinMode(BTN_PIN, INPUT_PULLUP);
    pixel.begin();
    pixel.setBrightness(50);
}

void loop() {
    if (digitalRead(BTN_PIN) == LOW) {
        pixel.setPixelColor(0, pixel.Color(0, 255, 0));   // Verde
    } else {
        pixel.setPixelColor(0, pixel.Color(255, 0, 0));   // Rojo
    }
    pixel.show();
    delay(50);
}
```

💡 **Explicación:** El botón con INPUT_PULLUP devuelve LOW al presionar. Según el estado, se establece el color y se llama pixel.show() para actualizarlo.

Cambio rápido en el quiz: Cambiar los colores: probar azul para suelto, amarillo para presionado.

Frase para el profesor: Combino digitalRead con el NeoPixel: el estado del botón determina el color del LED RGB.

15. NeoPixel cambia según potenciómetro

Posible pregunta: Cambiar el color del NeoPixel dependiendo del voltaje del potenciómetro.

Código completo:

```
#include <Adafruit_NeoPixel.h>

#define POT1_PIN     6
#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    pixel.begin();
    pixel.setBrightness(50);
}

void loop() {
    float voltaje = analogRead(POT1_PIN) * 3.3 / 4095.0;

    if (voltaje < 1.1) {
        pixel.setPixelColor(0, pixel.Color(0, 0, 255));   // Azul
    } else if (voltaje < 2.2) {
        pixel.setPixelColor(0, pixel.Color(0, 255, 0));   // Verde
    } else {
        pixel.setPixelColor(0, pixel.Color(255, 0, 0));   // Rojo
    }

    pixel.show();
    delay(100);
}
```

Explicación: El rango 0-3.3 V se divide en tres zonas: < 1.1 V → azul, 1.1-2.2 V → verde, >= 2.2 V → rojo. Se usa else if para evaluar en orden.

Cambio rápido en el quiz: Cambiar los umbrales 1.1 y 2.2 para ajustar el rango de cada color.

Frase para el profesor: Divido el rango del potenciómetro en tres zonas y asigno un color del NeoPixel a cada zona.

16. Monitor Serie como comando UART para LED externo

Posible pregunta: Recibir un carácter por Serial para controlar el LED externo. (e=encender, a=apagar, p=parpadear una vez)

Código completo:

```
#define LED_PIN 4

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    Serial.println("Listo. Comandos: e=encender, a=apagar, p=parpadear");
}

void loop() {
    if (Serial.available() > 0) {
        char cmd = Serial.read();

        if (cmd == 'e') {
            digitalWrite(LED_PIN, HIGH);
            Serial.println("LED encendido");
        } else if (cmd == 'a') {
            digitalWrite(LED_PIN, LOW);
            Serial.println("LED apagado");
        } else if (cmd == 'p') {
            digitalWrite(LED_PIN, HIGH);
            delay(300);
            digitalWrite(LED_PIN, LOW);
            Serial.println("LED parpadeado");
        }
    }
}
```

Explicación: Serial.available() indica si hay datos esperando. Serial.read() lee un carácter. Se compara con comillas simples ('e', 'a', 'p') porque es tipo char.

Cambio rápido en el quiz: Agregar nuevos comandos: otro else if con otro carácter. Cambiar el tiempo del parpadeo en delay(300).

Frase para el profesor: Serial.available() revisa si llegó un dato. Serial.read() lo lee. Comparo el carácter con los comandos definidos.

17. Monitor Serie como comando UART para NeoPixel

Posible pregunta: Recibir comandos por Serial para cambiar el color del NeoPixel. (r=rojo, g=verde, b=azul, o=apagar)

Código completo:

```
#include <Adafruit_NeoPixel.h>

#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    Serial.begin(115200);
    pixel.begin();
    pixel.setBrightness(50);
    pixel.show();
    Serial.println("Comandos: r=rojo, g=verde, b=azul, o=apagar");
}

void loop() {
    if (Serial.available() > 0) {
        char cmd = Serial.read();

        if (cmd == 'r') {
            pixel.setPixelColor(0, pixel.Color(255, 0, 0));
        } else if (cmd == 'g') {
            pixel.setPixelColor(0, pixel.Color(0, 255, 0));
        } else if (cmd == 'b') {
            pixel.setPixelColor(0, pixel.Color(0, 0, 255));
        } else if (cmd == 'o') {
            pixel.setPixelColor(0, pixel.Color(0, 0, 0));
        }

        pixel.show();
    }
}
```

Explicación: Misma lógica UART que el código anterior, pero ahora se controla el NeoPixel en lugar del LED. El pixel.show() se llama solo cuando se recibe un comando.

Cambio rápido en el quiz: Agregar más comandos: 'y' para amarillo (255,255,0), 'w' para blanco (255,255,255).

Frase para el profesor: Recibo el carácter por Serial y dependiendo del comando, establezco el color del NeoPixel y llamo show().

18. Integración: potenciómetros + OLED + LED externo

Posible pregunta: Mostrar los dos potenciómetros en la OLED y encender el LED si el potenciómetro 1 supera 2.0 V.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define LED_PIN   4
#define POT1_PIN  6
#define POT2_PIN  7

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    Wire.begin(8, 9);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
    float v1 = analogRead(POT1_PIN) * 3.3 / 4095.0;
    float v2 = analogRead(POT2_PIN) * 3.3 / 4095.0;

    // Controlar LED segun umbral
    if (v1 > 2.0) {
```

```
        digitalWrite(LED_PIN, HIGH);
    } else {
        digitalWrite(LED_PIN, LOW);
    }

    // Mostrar en OLED
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);

    display.setCursor(0, 0);
    display.print("Pot6: ");
    display.print(v1, 2);
    display.println(" V");

    display.setCursor(0, 16);
    display.print("Pot7: ");
    display.print(v2, 2);
    display.println(" V");

    display.setCursor(0, 32);
    display.print("LED: ");
    display.println(v1 > 2.0 ? "ON" : "OFF");

    display.display();
    delay(200);
}
```

Explicación: Integra tres periféricos: dos potenciómetros, OLED y LED. En el loop se leen, procesan y muestran todos juntos. El operador ternario (? :) decide qué texto mostrar.

Cambio rápido en el quiz: Cambiar el umbral de 2.0 a otro valor. Agregar más texto en la OLED con más setCursor y print.

Frase para el profesor: Leo ambos potenciómetros, controlo el LED según un umbral y muestro todo en la OLED en cada ciclo.

19. Integración completa: botón + potenciómetros + OLED + NeoPixel + LED

📋 **Posible pregunta:** Leer los potenciómetros, mostrar valores en OLED, indicar botón y cambiar el NeoPixel según el estado.

Código completo:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_NeoPixel.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define LED_PIN      4
#define BTN_PIN      5
#define POT1_PIN     6
#define POT2_PIN     7
#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);
    Wire.begin(8, 9);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    pixel.begin();
    pixel.setBrightness(50);
}

void loop() {
    float v1 = analogRead(POT1_PIN) * 3.3 / 4095.0;
    float v2 = analogRead(POT2_PIN) * 3.3 / 4095.0;
    bool presionado = (digitalRead(BTN_PIN) == LOW);

    // LED externo: encender si boton presionado
    digitalWrite(LED_PIN, presionado ? HIGH : LOW);

    // NeoPixel: segun voltaje del pot1
    if (v1 < 1.1) {
```

```
        pixel.setPixelColor(0, pixel.Color(0, 0, 255));    // Azul
    } else if (v1 < 2.2) {
        pixel.setPixelColor(0, pixel.Color(0, 255, 0));    // Verde
    } else {
        pixel.setPixelColor(0, pixel.Color(255, 0, 0));    // Rojo
    }
    pixel.show();

    // OLED
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);

    display.setCursor(0, 0);
    display.print("Pot6: "); display.print(v1, 2); display.println(" V");

    display.setCursor(0, 12);
    display.print("Pot7: "); display.print(v2, 2); display.println(" V");

    display.setCursor(0, 24);
    display.print("Boton: ");
    display.println(presionado ? "PRESIONADO" : "SUELTO");

    display.setCursor(0, 36);
    display.print("NeoPixel: ");
    if (v1 < 1.1)      display.println("Azul");
    else if (v1 < 2.2) display.println("Verde");
    else               display.println("Rojo");

    display.display();

    Serial.print("Pot6="); Serial.print(v1, 2);
    Serial.print(" Pot7="); Serial.print(v2, 2);
    Serial.print(" Btn="); Serial.println(presionado ? "ON" : "OFF");

    delay(200);
}
```

Explicación: Código completo que integra todos los periféricos. En setup() se inicializan todos. En loop() se leen todas las entradas y se actualizan todas las salidas. El Monitor Serie también muestra los valores para depuración.

Cambio rápido en el quiz: Este es el código base completo. Para el quiz, se puede simplificar quitando las partes que no se piden.

🗣️ **Frase para el profesor:** Este es el código de integración completa. En setup() configuro todo, en loop() leo entradas y actualizo salidas: OLED, NeoPixel, LED y Serial.

20. Código Base Recomendado para Memorizar

Este es el esqueleto completo. Úsalo como punto de partida en el quiz y quita lo que no necesites.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_NeoPixel.h>

// ── PINES ──────────────────────────────────────
#define LED_PIN      4
#define BTN_PIN      5
#define POT1_PIN     6
#define POT2_PIN     7
#define NEOPIXEL_PIN 48
#define NUM_PIXELS   1

// ── PANTALLA ───────────────────────────────────
#define SCREEN_WIDTH  128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ── NEOPIXEL ───────────────────────────────────
Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    // Serial
    Serial.begin(115200);

    // Pines digitales
    pinMode(LED_PIN, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);

    // OLED
    Wire.begin(8, 9);   // SDA=8, SCL=9
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.clearDisplay();
    display.display();

    // NeoPixel
    pixel.begin();
```

```
    pixel.setBrightness(50);
    pixel.show();
}

void loop() {
    // Leer entradas
    float v1 = analogRead(POT1_PIN) * 3.3 / 4095.0;
    float v2 = analogRead(POT2_PIN) * 3.3 / 4095.0;
    bool presionado = (digitalRead(BTN_PIN) == LOW);

    // Monitor Serie
    Serial.print("Pot6="); Serial.print(v1, 2);
    Serial.print(" | Pot7="); Serial.print(v2, 2);
    Serial.print(" | Btn="); Serial.println(presionado);

    // OLED
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.print("Pot6: "); display.print(v1, 2); display.println(" V");
    display.setCursor(0, 12);
    display.print("Pot7: "); display.print(v2, 2); display.println(" V");
    display.display();

    delay(200);
}
```

4. Lo Más Importante para Memorizar Antes del Quiz

⚠ **CLAVE:** Con INPUT_PULLUP, botón suelto = HIGH, botón presionado = LOW (porque se conecta a GND).

Líneas esenciales a memorizar:

```
// ── PINES ──────────────────────────────────────────
pinMode(4, OUTPUT);          // LED externo
pinMode(5, INPUT_PULLUP);    // Botón

// ── SALIDAS / ENTRADAS ─────────────────────────────
digitalWrite(4, HIGH);       // Encender LED
digitalWrite(4, LOW);        // Apagar LED

int boton = digitalRead(5);  // Leer botón (LOW = presionado)
int adc   = analogRead(6);   // Leer potenciómetro (0-4095)

float v = adc * 3.3 / 4095.0; // Convertir a voltaje

// ── SERIAL ─────────────────────────────────────────
Serial.begin(115200);
Serial.println("mensaje");
Serial.print(voltaje, 2);    // 2 decimales

// ── OLED I2C ───────────────────────────────────────
Wire.begin(8, 9);            // SDA=8, SCL=9

// ── BOTÓN CON INPUT_PULLUP ─────────────────────────
if (digitalRead(5) == LOW) {
    // Botón presionado
}
```

5. Lo que Más Probablemente Pueden Pedir Cambiar

Estos son los cambios más comunes que el profesor puede pedir durante el quiz:

Cambiar el tiempo de parpadeo

```
delay(500);  // Cambiar 500 por otro valor
```

📌 **Nota:** delay(200) = rápido | delay(1000) = lento | delay(2000) = muy lento

Cambiar el umbral de voltaje

```
if (voltaje > 2.0) {
    digitalWrite(LED_PIN, HIGH);
}
```

📌 **Nota:** Cambiar 2.0 por cualquier valor entre 0.0 y 3.3

Cambiar el texto de la OLED

```
display.println("Nuevo texto aqui");
```

📌 **Nota:** Solo modificar el string dentro de las comillas

Cambiar el color del NeoPixel

```
pixel.setPixelColor(0, pixel.Color(255, 0, 0));   // Rojo
pixel.setPixelColor(0, pixel.Color(0, 255, 0));   // Verde
pixel.setPixelColor(0, pixel.Color(0, 0, 255));   // Azul
pixel.setPixelColor(0, pixel.Color(0, 0, 0));     // Apagado
```

📌 **Nota:** Cambiar los tres valores R, G, B (0-255 cada uno)

Cambiar número de decimales

```
Serial.print(voltaje, 2);   // 2 decimales
Serial.print(voltaje, 3);   // 3 decimales
```

📌 **Nota:** El segundo parámetro controla los decimales

Cambiar entre ADC y voltaje

```
Serial.println(lectura);      // Muestra ADC (0-4095)
Serial.println(voltaje, 2);   // Muestra voltaje (0.00-3.30)
```

📌 **Nota:** Usar la variable que corresponde según lo que se pide

Cambiar el pin de un periférico

```
#define LED_PIN 4   // Cambiar 4 por el nuevo pin
```

📌 **Nota:** Si se define con #define, solo se cambia en un lugar

6. Sección de Repaso Rápido

Lee esto justo antes de entrar al laboratorio.

Función / Concepto	Significado / Uso
`setup()`	Se ejecuta una sola vez al encender. Aquí va: Serial.begin, pinMode, Wire.begin, display.begin, pixel.begin.
`loop()`	Se ejecuta repetidamente. Aquí va: digitalRead, analogRead, digitalWrite, display, pixel.show.
`pinMode(pin, OUTPUT)`	Configura el pin como salida. Usar para LED externo.
`pinMode(pin, INPUT_PULLUP)`	Configura el pin como entrada con resistencia interna. El botón conectado a GND dará LOW al presionar.
`digitalRead(pin)`	Lee HIGH o LOW. Con INPUT_PULLUP: suelto=HIGH, presionado=LOW.
`analogRead(pin)`	Lee el ADC. Devuelve 0-4095. No necesita pinMode.
`v = adc * 3.3 / 4095.0`	Fórmula para convertir lectura ADC a voltaje en V.
`Serial.begin(115200)`	Inicia la comunicación serie. Siempre en setup().
`Wire.begin(8, 9)`	Inicia I2C para la OLED. SDA=GPIO8, SCL=GPIO9.
`display.clearDisplay()`	Limpia el buffer de la OLED. Llamar antes de escribir.
`display.display()`	Envía el buffer a la pantalla. Siempre llamar al final.
`pixel.show()`	Actualiza el NeoPixel con el color definido. Siempre llamar después de setPixelColor.

☑ **Recomendación final:** No memorices el código completo. Entiende cada parte. Si sabes para qué sirve cada función y cómo se conecta cada periférico, puedes construir cualquier rutina que te pidan.